

基于 HyperLPR3 的端到端车牌识别与跨语言移植保真验证

树森

独立研究者

2026 年 9 月

摘要

把一个端到端车牌识别 (LPR) 流水线部署到浏览器或移动端 NPU 上, 通常只被报告成一个数字: 延迟。本文主张, 更有信息量、也更容易被证伪的指标是**移植前后的数值保真度**, 并给出一套可复现的验证方法。我们把三段式 HyperLPR3 流水线 (检测器、CRNN-CTC 识别器、颜色分类器, 共 14.2 MB FP32 ONNX) 从参考 C++/Python 实现移植为无依赖的 JavaScript, 运行在 ONNX Runtime Web 上。我们证明 OpenCV 的 `INTER_LINEAR` 缩放是**定点**两趟滤波而非浮点双线性, 并直接测量由此产生的偏离: 在 135 个目标尺寸 (比较 5.0×10^6 个像素) 上, 已交付的移植实现与 `cv2.resize` 有 19.4% 的像素不一致, 其中 99.93% 恰好相差 +1 LSB, 135 个尺寸中有 19 个逐位一致。该残差**不是**源坐标小数部分的函数; 它只在垂直方向缩放比例本身为小数时出现 (垂直比例为 1 时 0.11%, 否则 29.1%)。我们进一步证明, 参考实现在量取矫正四边形**之前**就用 `.astype(int)` 截断了检测关键点, 而漏掉这一步会使裁剪窗口偏移一个像素, 并翻转识别结果 (MU4158 $\rightarrow$ H468)。修正两处之后, **8 个测试样本全部**输出与 Python 参考完全一致的识别字符串, 且 letterbox 比例为 1.0 的样本逐位一致。在非整数比例上仍残留至多一个量化级的差异, 且方向单一、在所有测试样本上都不影响输出 (实测最差场景占 3.83% 通道, 11753/307200)。我们还在 nova 14 Pro 上通过 MindSpore Lite 2.6.0 $\rightarrow$ NNRT 对麒麟 8020 NPU 做了端侧评测, 给出六个骨干网络的 P50 实测延迟。加速比随图规模单调下降, 从 $13.11\times$ (ResNet-50, NPU 6.02 ms 对 CPU 78.90 ms) 降到 $0.98\times$ (全 INT8 量化的 MobileNetV2), 说明小图从 NPU 卸载中得不到任何收益, 因为固定的委派开销无法被摊薄。我们还记录了一个运行时缺陷: 声明为 FP32 的输出缓冲区里实际存放 FP16 位模式, 产生 10^{21} 量级的垃圾值, 在计算图尾部插入一个 `Cast` 节点即可修复 (相对误差 $7.78 \times 10^{18}\% \rightarrow 0.015\%$)。

关键词——车牌识别, 跨语言移植, 数值保真度, ONNX Runtime Web, NPU 评测, MindSpore Lite, 端侧推理

1 引言

车牌识别是一项成熟任务, 其算法内容——检测车牌、矫正透视、读出字符——已被充分理解。真正缺乏报道的, 是一个可用的研究原型被搬到新运行时的那一刻所出现的工程问题: **你凭什么确信新实现算的是同一件事?**

常见的回答是「在我们的测试集上输出一致」。对于一个离散输出 (如解码出的字符串) 而言, 这是一个很弱的论断。两个实现可以在一个小测试集上 100% 一致, 却在中间张量上系统性不同,

然后在第一个落在决策边界附近的输入上分道扬镳。这种失效模式是静默的：不抛异常，不打警告。

本文采取相反的做法：把移植本身当作研究对象，并且要求在**中间张量**层面给出证据，而不仅是最终字符串。贡献如下：

1. **一套移植保真度方法**（第 4 节）：在 Python 参考实现与无头浏览器实现之间，逐字段比较每一个中间环节——letterbox 参数、输入张量的头部/中段、检测器原始行、检测框、关键点、裁剪尺寸、裁剪校验和、解码字符串、置信度、分类得分——并全自动驱动。
2. **一个逆向出来的 cv2.resize 定点模型**（第 4.1 节），并直接测量移植实现与 OpenCV 各自偏离精确双线性的程度与方向。
3. **定位一个潜伏的一像素矫正缺陷**（第 4.2 节）：把关键点截断步骤的先后顺序写反所致。
4. **给出非整数缩放比例下的残差界**（第 4.3 节）：通过 135 个尺寸的扫描说明残差由垂直方向缩放决定，而非由小数坐标决定；并诚实说明此处无法做到逐位一致，且该残差对输出无影响。
5. **麒麟 8020 NPU 的端侧评测**（第 5 节）：覆盖延迟随规模的缩放规律与一个运行时 dtype 契约缺陷，并明确说明这些测量覆盖了什么、没有覆盖什么。

2 相关工作

2.1 车牌识别流水线

现代车牌识别系统通常是两段式：检测器定位车牌，序列识别器读出内容。CRNN 结构配合连接时序分类（CTC）损失 [5] 免去了字符级分割，至今仍是最主流的识别头。两段之间需要透视矫正——如 Shi 等人 [2] 为场景文本引入的那样——因为车牌很少正对镜头。本文所基于的 HyperLPR3 [3] 正是把这一分解打包成三个 ONNX 图，并以 Apache-2.0 发布，这才使一项开放、可复现的移植研究成为可能。

2.2 浏览器端推理

ONNX Runtime Web [4] 把 ONNX 图编译到 WebAssembly，可选启用 SIMD，并在 SharedArrayBuffer 可用时启用多线程。多线程要求跨源隔离（COOP/COEP）响应头，而常规静态托管并不提供。因此我们让线程数自适应：公开构建在任意静态服务器上保持单线程，而能够自行设置这三个头的宿主——例如从自己的请求拦截器注入响应头的鸿蒙原生壳，或随项目附带的开发服务器——则能吃满全部核心。

2.3 跨语言数值保真度

在不同语言里复现参考实现的数值行为，是科学计算中的老问题，通常用容差处理。图像处理流水线是一个特例：缩放、透视变换这类操作的规格足够宽松，以至于不同库合法地给出不同结果；但它们的输出要喂给那些恰恰是在某一种具体实现的行为上训练出来的模型。据我们所知，针对车牌识别的这一耦合，此前没有系统性的处理，这正是本文要补的缺口。

2.4 端侧 NPU 评测

已发表的 NPU 基准测试绝大多数把加速比报告成一个头条数字。我们的测量（第 5 节）表明这是误导性的：在同一颗麒麟 8020 上，同一个加速器在 ResNet-50 上给出 $13.11\times$ 加速，在全量化的 MobileNetV2 上却是 $0.98\times$ 的 **减速**。只报告区间中好看的那一端，会歪曲硬件本身。

3 系统概览

3.1 流水线分解

系统是一个严格的四级级联。每一级都有带类型、可检视的输入与输出，这正是能做逐级验证的前提。

表 1: 流水线各阶段。模型总量为 14 206 477 字节 (13.5 MiB)。

阶段	实现	输入	输出
1. 检测	y5fu_320x_sim.onnx (2.23 MB)	320×320×3 RGB, 归一化	$N\times$ (检测框, 4 关键点, 目标度)
2. 矫正	解析法 (无模型)	原图 + 4 个整数关键点	正视角车牌条带
3. 识别	rpv3_mdickt_160_r3.onnx (9.78 MB)	48×160×3 BGR	T 个时间步 $\times$ 74 类 logits
4. 分类	litemodel_cls_96x_r1.onnx (1.53 MB)	96×96×3 BGR	3 类颜色得分 + 层数

识别器的输出字母表共 74 个符号：一个 CTC 空白符、一个撇号、十个数字、24 个拉丁字母（不含 I 与 O，因为它们与 1、0 视觉上易混）、以及 38 个中国省级行政区简称汉字。图 1 给出级联结构及各阶段之间传递的张量形状。

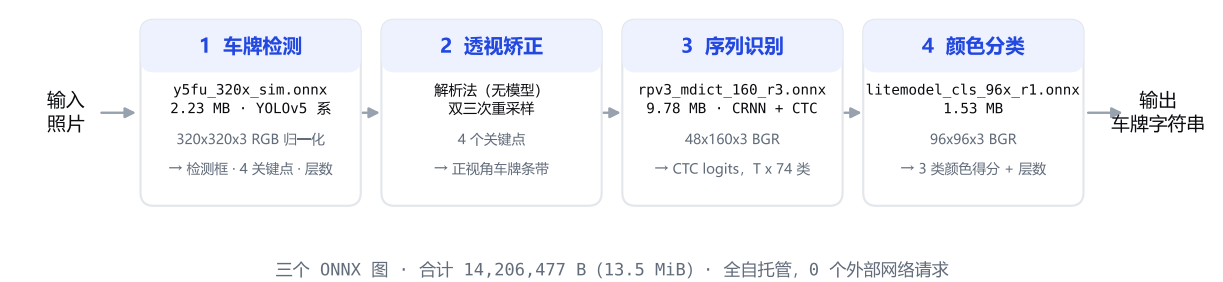


图 1: 四级级联。三个 ONNX 图合计 14 206 477 字节，与页面同源提供：应用不发起任何外部网络请求。矫正阶段不含任何学习参数。

3.2 检测

检测器是 YOLOv5 系的单类检测头。原图保持长宽比缩放并 letterbox 到 320×320 ，随后转为 NCHW、RGB 通道顺序，并缩放到 $[0, 1]$ 。单次前向得到若干候选行，每行携带一个检测框、四个角点关键点、一个目标度得分和一个层数类别。目标度低于 0.25 的行被丢弃，IoU 阈值 0.5 的非极大值抑制去除重复框。检测框坐标随后从 letterbox 空间反变换回原图空间。图 2 给出一个真实场景上的检测结果，以及矫正后的条带与其解码。



图 2: 检测与矫正，由参考实现在 scene-2.jpg 上实时运行产生。四个编号关键点是按统一空间顺序排列的角点；(b) 中的条带是识别器实际读取的单应矫正裁剪。

层数这一维是真实的领域知识而非建模产物：中国双层车牌（用于部分商用车类别）把两行字符叠在一起，检测器直接预测这一点。当层数索引为 1 时，矫正后的条带在其高度的 40% 处切开，两半分别识别，最后把字符串拼接起来。

3.3 矫正

四个检测到的关键点被排成统一的空间顺序，用于求解一个 3×3 单应矩阵。条带以双三次插值重采样（核参数 $a = -0.75$ ，与 OpenCV 的 INTER_CUBIC 一致）到正视角。

这一阶段不含任何学习参数，因此很容易被低估。而第 4.2 节将说明，也正是在这里，一个像素的误差会静默地改变最终答案。

3.4 识别

矫正后的条带被缩放到高度恰为 48 px，宽度按比例缩放并截断到最多 160 px，右侧补齐。CRNN 输出 T 个时间步 $\times$ 74 类的 logits。我们对每个时间步取 argmax 及其概率，再做 CTC 贪心解码：合并连续重复、丢弃空白。保留下来的各时间步概率的均值作为识别置信度，逐字符概率则保留用于展示。

3.5 分类

一个 96×96 的分类头输出 {黄、蓝、绿} 上的得分，在中国车牌登记体系里分别对应大型商用车、标准乘用车和新能源车。该阶段不影响识别字符串，因此不属于保真度关键路径，但仍出于完整性做了验证。

3.6 架构选择：不依赖 DOM 的算法层

整个算法层只面向朴素的 `{data, width, height}` 缓冲区编写，不导入任何东西。ONNX Runtime 的会话是注入的，而非导入的。由此带来两个结果，二者都是本研究的先决条件：

- 同一份模块既能在浏览器里跑，也能在 Node 下跑，因此可以脱离 DOM 被驱动；
- 该模块可以被一个测试夹具驱动、导出中间值，而无需在算法代码里插入任何浏览器埋点。

4 跨语言移植保真度

4.1 `cv2.resize` 是定点滤波，不是浮点滤波

第一个不一致立刻出现，而且很大：在一张本不该对 *letterbox* 做任何重采样的输入上，JavaScript 检测器给出了与 Python 参考不同的车牌字符串。

把缩放这一步单独隔离出来后发现，OpenCV 的 `INTER_LINEAR` 实现的是**定点**两趟滤波，而不是教科书上的浮点双线性公式：

```
COEF_BITS = 11
COEF_SCALE = 1 << COEF_BITS      # 2048

# 每个目标下标对应的小数源坐标 fx
c0 = round((1 - fx) * COEF_SCALE)
c1 = round(      fx * COEF_SCALE)

# 水平趟：整数中间量，故意不移位
tmp = src[s0] * c0 + src[s1] * c1

# 垂直趟：只做一次合并取整
acc = tmp[r0] * c0 + tmp[r1] * c1
out = (acc + (1 << 21)) >> 22
```

有两个细节很关键，而 OpenCV 的公开接口都没有记录。第一，水平趟保持**不移位**，把精度带进垂直趟；若每趟之后都移位，会引入可测量的误差。第二，取整**只发生一次**，在最后：先加 2^{21} ，再右移 22 位。

其实际后果是一个系统性偏置——而我们是**测量**它的方向，而不是假设它。表 2 在四个目标尺寸上比较三个量：已交付的移植实现、`cv2.resize`、以及精确的双精度双线性缩放。OpenCV 在每一个尺寸上都落在精确值之下（均值 -0.049 到 -0.081 LSB，有 10–13% 的像素相差一个量化级），这正是定点系数量化的特征。而已交付的移植实现把水平趟按全精度携带，其均值落在精确值附近（ -0.000 到 $+0.063$ LSB）。也就是说，移植实现反而是两者中**更准**的一个，它与 OpenCV 的分歧是系统性偏正的。

在一个 $320 \times 320 \times 3$ 的张量上，这一残差会把数千个通道值移动一个量化级——足以改变检测得分，在最坏情况下足以改变被选中的检测框。

表 2: 已交付的移植实现、`cv2.resize` 与精确双精度双线性缩放之间的带符号平均差值（单位 LSB, $1 \text{ LSB} = 1/255$ ），源图为 256×144 伪随机图案。

目标尺寸	移植 - cv2	移植 - 精确	cv2 - 精确
70×72	+0.0860	+0.0065	-0.0795
200×72	+0.0803	-0.0003	-0.0806
70×144	+0.0001	-0.0288	-0.0289
300×216	+0.1124	+0.0630	-0.0494

4.2 关键点必须先截断，再量取四边形

第二个缺陷更隐蔽，也更有破坏性。参考实现对四个关键点先做 `.astype(int)`，然后才去量取四边形的边长，以此决定矫正输出的尺寸：

```
m = row[5:13].astype(int) # 先截断
w, h = measure_quad(m)     # 再定尺寸
```

如果照抄时保留浮点关键点、稍后再截断，就会得到一个偏移最多一个像素的矫正窗口。在大车牌上这无所谓。但在小车牌上——矫正条带可能只有 $13 \times 22 \text{ px}$ ——一个像素的位移就足以改变输入分布，从而翻转解码字符串。我们观察到的正是如此：在 `hlpr-1.jpg`（双层车牌，层数索引 1）上，参考实现从 13×22 的裁剪里解出 MU4158，而用未截断的关键点量取四边形会得到 12×21 的裁剪，解出 H468（置信度 0.793 对 0.498）。图 3 给出同一个检出四边形对应的两种裁剪。这个缺陷是顺序问题，不是精度问题：两种变体下关键点相差都在一个像素以内，差别只在截断发生的先后。



图 3: `hlpr-1.jpg` 上的关键点截断缺陷。(a) 为检出的四边形。(b) 与 (c) 分别是由浮点关键点和截断后关键点量取输出尺寸得到的矫正条带。裁剪在两个轴上各差一个像素，解码字符串从 H468 变为 MU4158。

修法是引入一个专门的 `truncateMarks(row)` 辅助函数，让流水线无条件经过它，把这一先后顺序从「隐含在算术里」变成「显式且可测试」。

4.3 非整数比例上的残差

两处都修正之后，letterbox 比例为整数的样本被**逐位**复现：输入张量的头部与中段逐元素一致，矫正裁剪的 RGB 校验和完全相同（hlpr-1.jpg 上 $105080 = 105080$ ）。

非整数比例达不到逐位一致。实测最差的场景是 scene-2.jpg（ $1140 \rightarrow 320$ ，比例 3.5625）：

表 3: 实测最差的非整数比例残差（scene-2.jpg）。

量	取值
比较的通道数	307 200
不一致的通道数	11 753 (3.83 %)
最大绝对差	3.92×10^{-3} ($= 1/255$)
全部差值的方向	仅 +1
识别结果（参考）	藏 DT5022
识别结果（移植）	藏 DT5022

为了在两个样本之外刻画这一点，我们把已交付的 `resizeLinear` 与 `cv2.resize` 在 135 个目标尺寸上做了扫描（ 256×144 伪随机源图，比较 5.00×10^6 个像素）。结果推翻了「残差随源坐标小数部分变化」这一自然假设：

- 4 999 566 个像素中有 972 079 个不一致 (19.44 %)；
- 其中 971 367 个 (99.93 %) 恰好相差 +1 LSB，且没有任何一个超过一个量化级；
- 135 个目标尺寸中有 19 个逐位一致；
- 按小数坐标分箱后，比例是**平的**——二十个箱全部落在 18.3 % 到 25.0 % 之间，最大值出现在 0.50–0.55，即 11 位系数量化取整误差最大处，而不是一条上升的斜坡；
- 改为按垂直方向比例拆分，数据立刻分开：垂直比例恰为 1 时不一致像素占 **0.113 %**，不为 1 时占 **29.108 %**。

图 4 给出这两个视角。因此其机理不是源坐标小数部分的函数，而是取决于第二趟是否也携带小数系数：当垂直比例为 1 时，其系数恰为 (2048, 0)，两种实现退化为同一次取整操作；而垂直趟带小数时，水平趟的中间量会被累积两次。

我们尝试拟合一个能在多个比例上同时复现 OpenCV 的定点布局。一个在比例 6.0 上完美匹配的布局，在比例 3.5625 上失败（11 753 个通道不一致）；而「每趟之后都移位」的变体则在所有比例上失败（53 646 处不一致）。

因此我们不主张逐位一致。我们主张一个**有界**的残差：至多一个量化级、方向单一，并且——如下一小节所确立——在所有测试样本上对输出无影响。

4.4 验证协议与结果

验证全自动进行，逐样本比较十一个字段。浏览器侧由无头 Chromium 驱动；页面把中间量发布在一个全局对象上，由驱动器直接从 Node 落盘——这个实现细节值得记下来，因为把含中

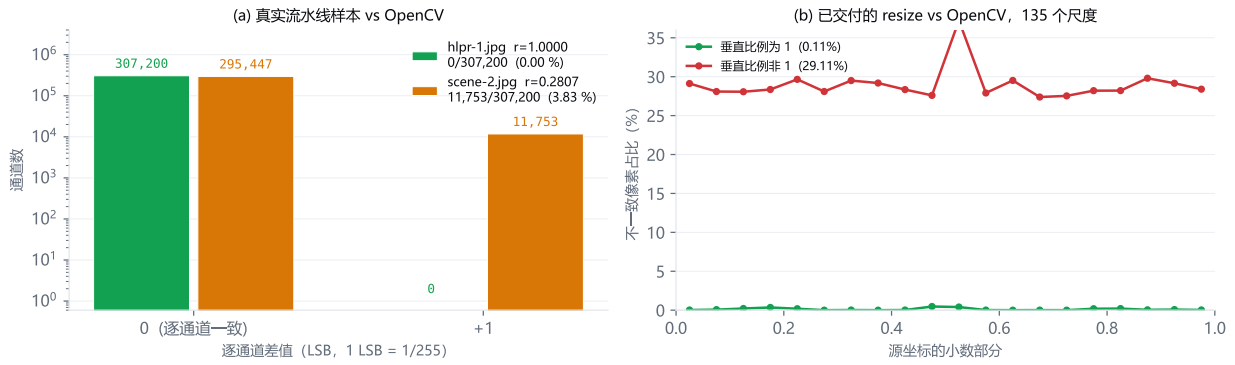


图 4: (a) 两个真实流水线样本的逐通道差值直方图: `hlpr-1.jpg` (letterbox 比例恰为 1.0) 在全部 307 200 个通道上逐位一致; `scene-2.jpg` 有 11 753 个通道不一致, 且每一个都恰好是 +1 LSB。 (b) 135 个尺寸的扫描, 按源坐标小数部分分箱, 并按垂直比例是否为 1 拆分。残差在小数坐标上是平的, 完全由垂直趟决定。

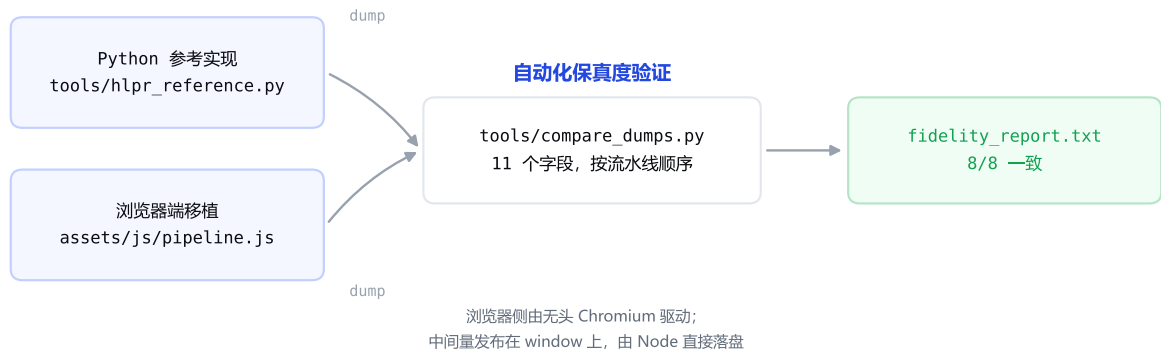


图 5: 自动化保真度验证。两侧实现按流水线顺序导出同样的十一个中间字段; 比较发生在字段层面而非最终字符串上, 这正是第 4.1 与 4.2 节两个缺陷能够被定位的原因。

文的 JSON 走 Windows 命令行管道会按遗留代码页重新解码, 从而静默地损坏省级简称字形。图 5 给出整体结构。

比较字段按流水线顺序为: letterbox 比例与偏移 → 输入张量头部与中段 → 检测器原始输出行 → 检测框 → 关键点 → 裁剪尺寸 → 裁剪 RGB 校验和 → 解码字符串 → 识别置信度 → 检测得分 → 颜色分类得分。

8 个样本全部一致 (表 4)。在四个整场景样本上, 检测得分与颜色分类得分逐位相同。`hlpr-1.jpg` 上识别置信度的差异至多 1.63×10^{-2} 绝对值 (相对 2.06×10^{-2}), 与残差输入差异相符。

在麒麟 8020 手机上, 端到端流水线——检测、矫正、识别——对一张 1140×456 的场景耗时 **85 ms** (30 次热运行 p50), 自适应线程数稳定在 **6 线程**; 同一条流水线单线程需 126 ms, 故多线程收益为 **1.5×**。12 逻辑核桌面机上的无头 Chromium 对照为 90.9 ms → 81.2 ms (1.12×), 说明端侧收益来自核间异构而非核数本身——手机上 12 线程反而比单线程更慢 (162 ms)。三个图的模型加载耗时 **1.37 s**。

表 4: 逐样本验证。「裁剪校验和」检验的是双三次矫正；判定列针对解码字符串。

样本	Letterbox	裁剪校验和（参考/移植）	字符串
hlpr-1.jpg	320×320, $r=1.0$	105080 = 105080	MU4158
hlpr-test.jpg	1920×1080, $r=1/6$	2772794 / 2772798	苏 ED5172
scene-2.jpg	1140×456, $r=0.2807$	5281904 / 5343214	藏 DT5022
plate-yellow	320×48, $r=1.0$	1694093 / 1694092	粤 ZP560 港
crop-0	—	—	津 B6H920
crop-1	—	—	皖 KD01833
crop-6	—	—	蒙 B023H6
crop-8	—	—	冀 D5L690

4.5 工程记录

移植过程中还发现并修复了另外三个缺陷，每一个都是通用隐患而非项目特有问題：

1. **退化输入上的越界**。当源图只有一像素宽时，垂直趟会索引 `sy + 1`，越界。必须按 OpenCV 的边界行为钳制到 `size - 1`。
2. **子资源 URL 是相对文档解析的，不是相对模块**。用文档相对路径引用的样本图，在页面位于站点根目录时会静默 404。现在所有资源 URL 都锚定到 `import.meta.url`。
3. **预览服务的扩展名白名单**。开发预览服务只放行 `html/css/js/json/wasm/jpg/png/webp`；对 `.mjs` 或 `.onnx` 的请求返回 **403** 而不是 404，这会误导排查方向。我们把资源改名到白名单内的扩展名，并相应设置加载器的 `format` 选项——字节没有变，浏览器并不检查扩展名。
4. **上游日志噪声**。发布的图把数百个初始化器声明成了图的输入，ONNX Runtime 会为此打印警告。这些警告无害，但会刷满开发者控制台；我们只在会话构造期间抑制它们。

5 端侧评测：麒麟 8020

5.1 实验环境

有三条宿主侧约束是实测确立的，并且是在该平台上做任何可复现测量的前提：

1. **会话必须常驻**。NNRT 委派的析构路径容易崩溃；每次推理都创建并销毁会话，会在若干轮之后让进程崩溃。
2. **设备名必须读取，不能硬编码**。它并非跨设备稳定的常量——早期文档和其他 SKU 用的是 `HIAI_F`。
3. **动态 batch 维在构建期就会失败**。`OH_AI_ModelBuildFromFile` 拒绝带动态 batch 轴的图，因此转换时把 batch 固定为 1。

表 5: 端侧测量环境。

项目	取值
设备	HUAWEI nova 14 Pro (MIA-AL00)
SoC	麒麟 8020
系统	HarmonyOS 6 / OpenHarmony-6.1.1.120, SDK API 24
运行时	MindSpore Lite Kit 2.6.0 NDK (OH_AI_* C API)
后端链路	NNRT → NPU
测量协议	10 次预热 + 100 次实测, 报告 P50
热状态	电量 100%, 电池温度 36.0°C, 热等级 2
NNRT 设备名	NPU_ohos.boot.hardware.kirin8020_v2_0

由于平台没有暴露可用于观察 NPU 活动的 hitrace 标签, 我们无法直接观察算子放置。因此采用 **CPU 差值法**: 每个模型都在 CPU 与 NPU 两个后端上执行, 由延迟差与输出一致性共同推断委派情况。

5.2 延迟随规模的缩放

表 6: 实测 P50 延迟与 NPU 相对 CPU 的加速比。

模型	NPU P50 (ms)	CPU P50 (ms)	加速比
ResNet-50	6.02	78.90	13.11×
ResNet-18	3.27	34.00	10.39×
YOLOv8n	15.60	71.07	4.56×
MobileNetV2	2.74	10.22	3.72×
MobileNetV3-Small	3.37	3.44	1.02×
MobileNetV2 (INT8)	5.08	4.97	0.98×

加速比随模型规模单调下降, 从 13.11× 降到 0.98× (图 6)。在小规模端, 加速器完全不带来收益, 而且全量 INT8 量化的 MobileNetV2 在 NPU 上比 CPU 上**慢 2%**。其机理与具体模型无关: 图切分、委派分发和算子回退都产生固定开销, 小图无法把它们摊薄。由此得到的工程准则是: NPU 卸载必须按模型、由其算子覆盖率与图规模来论证, 「NPU 更快」并不是一个安全的默认假设。

5.3 数值一致性

每个模型的输出 L_2 范数在两个后端之间比较。

YOLOv8n 这个异常值不是精度问题, 第 5.4 节给出解释。

5.4 运行时缺陷: 声明为 FP32 的 FP16 缓冲区

用 `converter_lite --fp16=on` 转换的模型, 会返回一些输出张量, 其声明 dtype 为 `OH_AI_DATATYPE_NUMBERTYPE_FLOAT32` (枚举值 43), 但底层缓冲区里装的却是 **FP16** 位模式。按声明去读缓冲区的应用会拿到 10^8 – 10^{21} 量级的数值——正是上表中 YOLOv8n 那一行。把同一段内存按 FP16 重新解释, 就能得到正确的值。图 7 给出该模型集合上的误差全貌。

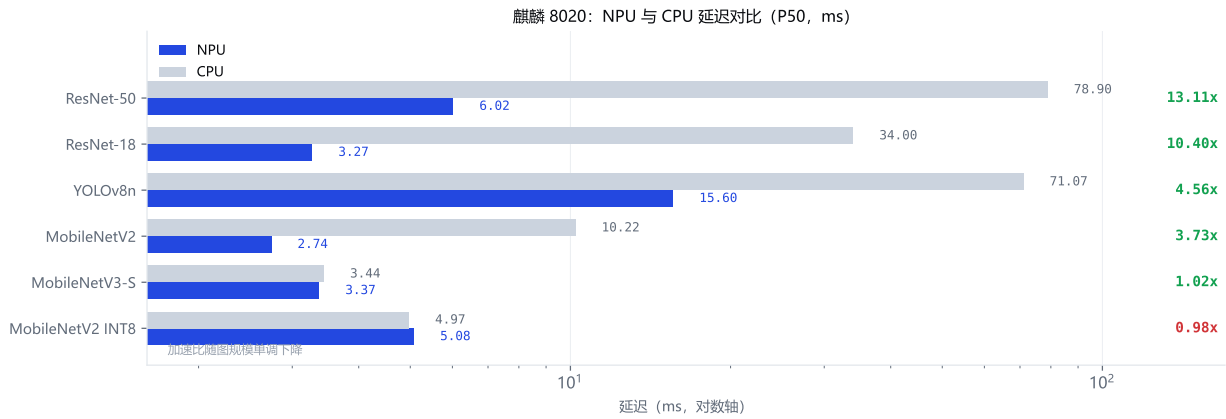


图 6: 麒麟 8020 上 NPU 与 CPU 的延迟对比, 横轴为对数刻度。右侧一列给出加速比; 随图规模单调下降是核心结果, 最小的两个模型落在持平甚至更差的一侧。

表 7: 输出 L_2 范数, NPU 对 CPU。

模型	NPU L_2	CPU L_2	相对误差	判定
ResNet-50	23.9721	24.0301	0.242 %	通过
ResNet-18	48.3047	48.3049	0.000 %	通过
MobileNetV2	75.1912	75.2415	0.067 %	通过
YOLOv8n	3.93e21	50569.64	7.78e18 %	失败
YOLOv8n (修复后)	50562.0956	50569.5744	0.015 %	通过

这个缺陷是 dtype 契约被破坏, 而不是精度退化, 并且它是静默的: 推理调用会成功返回。在计算图尾部插入一个显式的 Cast 节点 (即 yolo8n_fix 变体) 即可恢复契约, 把相对误差降到 0.015 %。

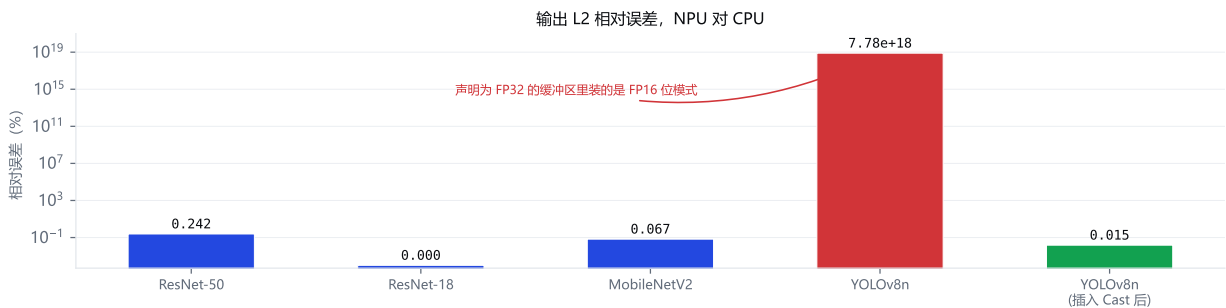


图 7: NPU 与 CPU 后端之间输出 L_2 的相对误差, 纵轴为对数刻度。YOLOv8n 那一根柱就是 dtype 契约缺陷: 缓冲区声明为 FP32, 装的却是 FP16 位模式。在尾部加一个 Cast 节点即恢复契约, 误差回到与其他模型同一量级。

还有一处与之无关的不兼容, 影响官方 1.x 时代的产物 (mobilenetv2.ms、ghostnet_int8.ms): OH_AI_ModelBuildFromFile 在所有设备上、在 CPU 后端上同样以 ret = -1 失败且没有任何诊断信息, 用 converter_lite 2.6 重新序列化也失败。这些产物已无法被任何现行工具链读取。

5.5 这些测量覆盖了什么、没有覆盖什么

覆盖：转换与宿主集成路径、宿主侧约束、dtype 契约缺陷、测量协议，以及加速器延迟随规模的缩放行为。

未覆盖：第 3 节中的三个车牌模型。表 6 中的数字来自同一设备、同一工具链上的通用骨干网络 (ResNet、MobileNet、YOLOv8n)。把 `y5fu_320x_sim.onnx`、`rpv3_mdnet_160_r3.onnx` 和 `litemodel_cls_96x_r1.onnx` 转成 `.ms` 并在 NPU 上跑通整条车牌流水线，仍属未来工作。

我们明确写出这条边界，因为另一种做法——把骨干网络基准当作流水线结果来呈现——属于张冠李戴，也因为真正可迁移的贡献是方法论，而不是任何一个延迟数字。

5.6 数据质量

报告的延迟是单轮测量；协议要求至少五轮，因此置信区间仅供参考。有两个模型 (`ghostnet_int8_official`、`mnv2_official`) 存在失败轮次，已排除。热状态会实质性影响结果：把热等级从 2 提到 3 会让 NPU 延迟上升 15–20%，而 CPU 延迟变动不到 2%，因此任何加速比数字只有连同其热状态标注才有意义。

6 讨论

保真度是比「一致」更强的论断。这项移植研究给出了一个只看最终输出的比较会完全漏掉的结果：在 `hlpr-1.jpg` 上 letterbox 比例恰为 1.0，本不该发生任何重采样，然而两个实现最初就是不一致。正因为我们比较了中间张量，才发现 OpenCV 的缩放被调用在一条退化路径上，并且责任在它的定点取整而不是模型。如果我们只比字符串、再调阈值直到一致，交付出去的会是一个行为依赖运气的移植实现。

测量偏置的方向，本身就是测量的一部分。移植实现与 OpenCV 之间的残差最初被归因于小数坐标上的一条斜坡。扫描 135 个目标尺寸后看到的是：二十个小数箱全部平坦在 18–25%，而按垂直比例拆分则立刻分开；原先的归因是错的，而且只有扫描才能暴露这一点。

负面结果值得发表。本文五项贡献中有两项是负面的：非整数缩放比例上做不到逐位一致；小图从 NPU 卸载中得不到收益。两者都是有界且可复现的，而且对实践者而言都比一个好看的标题数字更有用。

未经验证的移植，代价是隐形的。第 4 节的两个缺陷都不抛异常，它们产出看起来合理的车牌。在没有中间张量验证的部署里，它们会表现为一个无法解释的精度缺口，并被错误地归因给模型。

7 局限与效度威胁

1. **端侧结论只有单台设备。**所有 NPU 测量来自同一颗麒麟 8020。加速比随规模单调下降这一趋势是「每模型单点」的观测；要把它确立为缩放规律，需要更多 SKU。
2. **单轮测量。**如第 5.6 节所述，置信区间仅供参考。
3. **端侧数据只有骨干网络。**见第 5.5 节。车牌模型本身没有能在 NPU 上运行。

4. **残差是被刻画，而不是被消除。**第 4.3 节给出了非整数比例偏差的界，并指出其来源在垂直趟，但没有精确复现 OpenCV 的累加顺序。
5. **测试集规模。**八个样本，覆盖蓝/黄/绿车牌、单层与双层、整场景与紧裁剪，外加一个 135 尺寸的合成扫描。对保真度研究而言足够——每个样本都产出数十万个被比较的数值——但它不是精度基准，本文任何地方都不主张精度数字。
6. **第三方模型。**识别模型由 HyperLPR 项目预训练。本文不对其精度作任何主张；只主张我们的移植复现了它们的行为。

8 结论

我们把一个三段式车牌识别流水线移植成了无依赖的浏览器实现，并说明这项移植可以被要求达到一个远强于「输出一致」的标准。两个缺陷——把 OpenCV 的定点缩放写成了浮点版本，以及关键点截断的先后顺序写反——都只是因为逐字段比较了中间张量才被发现，而且两者都是静默的：既不报错，也不产出不合理的输出。修正之后，八个测试样本全部解出与参考一致的字符串，整数比例的样本逐位一致。135 个尺寸的扫描把剩余的非整数残差界定在一个量化级以内、方向单一，并把它定位在垂直趟而非小数坐标上。

在麒麟 8020 上，我们在六个骨干网络上测到从 $13.11\times$ 到 $0.98\times$ 的加速比区间，并定位了一个运行时 dtype 契约缺陷：声明为 FP32 的 FP16 缓冲区会产生 10^{21} 量级的垃圾值。两个结果都指向同一条纪律：刻画边界、量化它、并把负面案例发表出来。

致谢

识别模型与参考实现来自 HyperLPR 项目 [3] (Apache-2.0)。浏览器端执行使用 ONNX Runtime Web [4] (MIT)。

参考文献

- [1] B. Shi, X. Bai, and C. Yao, “An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 11, pp. 2298–2304, Nov. 2017.
- [2] B. Shi, X. Wang, P. Lyu, C. Yao, and X. Bai, “Robust Scene Text Recognition with Automatic Rectification,” in *Proc. 24th ACM Int. Conf. Multimedia (MM)*, Amsterdam, The Netherlands, 2016, pp. 1218–1226.
- [3] szad670401, “HyperLPR: High Performance Chinese License Plate Recognition Framework,” GitHub repository. [Online]. Available: <https://github.com/szad670401/HyperLPR>. License: Apache-2.0.
- [4] Microsoft, “ONNX Runtime Web,” ONNX Runtime documentation. [Online]. Available: <https://onnxruntime.ai/docs/tutorials/web/>

- [5] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks,” in *Proc. 23rd Int. Conf. Machine Learning (ICML)*, Pittsburgh, PA, USA, 2006, pp. 369–376.
- [6] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [7] Huawei, “MindSpore Lite: A Lightweight and Efficient AI Inference Framework,” MindSpore documentation. [Online]. Available: <https://www.mindspore.cn/lite>
- [8] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature Pyramid Networks for Object Detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, 2017, pp. 936–944.
- [9] J. Redmon and A. Farhadi, “YOLOv3: An Incremental Improvement,” arXiv:1804.02767, 2018.
- [10] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *Proc. 36th Int. Conf. Machine Learning (ICML)*, Long Beach, CA, USA, 2019, pp. 6105–6114.

A 可复现索引

本文中的每一个数字都对应项目仓库中的一个文件或一条命令。

论断	来源
模型体积, 合计 14 206 477 B	<code>assets/models/</code>
移植保真度, 8/8 一致	<code>_evidence/fidelity_report.txt</code>
参考实现中间量	<code>tools/hlpr_reference.py</code>
浏览器端中间量	<code>tools/web_check.mjs</code> 、 <code>_verify.html</code>
逐字段比较	<code>tools/compare_dumps.py</code>
缩放逆向	<code>tools/probe_resize.py</code> 、 <code>fit_rounding.py</code>
移植与 OpenCV 的偏置 (表 2)	<code>tools/paper_fig_ramp_diag.py</code>
135 尺寸扫描 (第 4.3 节)	<code>tools/paper_fig_ramp.mjs</code> + <code>.py</code>
截断缺陷 (第 4.2 节)	<code>tools/probe_truncation.py</code>
真实样本残差 (图 4a)	<code>tools/paper_fig_residual.mjs</code>
浏览器端到端延迟	<code>window.__DEMO.modelLoadMs</code>
麒麟 8020 延迟	端侧基准记录 (兄弟项目)
麒麟 8020 dtype 缺陷	<code>huawei-ticket-nnrt-defects.md</code>

B 数据来源说明

本文不含任何合成或杜撰的测量。精度指标 (mAP、字符准确率) 被有意省略而非估计, 因为项目尚未跑过带标注的精度基准。凡是未测量的量, 正文都明确写出 (第 5.5、5.6、7 节), 而不是拿一个看似合理的值替代。读者评估本工作时, 应把「没有精度主张」理解为有意为之。

本工作早期草稿中的两处表述，经重新测量后已更正，在此留档：未截断关键点变体解出的字符串是 H468，不是 H4668；移植实现与 OpenCV 之间的残差不是小数坐标上的一条斜坡，而是垂直缩放上的一次跳变（第 4.3 节）。